YAS IT-FACTORY · Fiches de temps — Documentation du projet

Oct 5, 2026 · @Cheikh Thioune

Synthèse

L'application remplace le fichier Excel refait à la main chaque vendredi et chaque fin de mois. Le cœur du produit est **terminé et testé en local** : saisie quotidienne, rapports hebdo et mensuel au format YAS, validation, envois automatiques. La **liaison GitLab** est codée et testée avec un faux GitLab, mais **pas encore branchée sur it-gitlab.yas.sn**. Avant la mise en production, il reste surtout de l'exploitation : base PostgreSQL, SMTP Office 365, secrets, sauvegardes et comptes réels.

Domaine	État	Commentaire
Saisie quotidienne et tâches	Fait	Consultant, PM, Head of Office, Admin
Rapport hebdo (lundi → dimanche) et fiche mensuelle (20 → 19)	Fait	Excel au format exact de la fiche YAS
Soumission, validation, renvoi pour correction	Fait	Verrouillage des saisies une fois soumis
E-mails et relances automatiques	Fait	Testés en mode « outbox » (fichiers locaux), pas avec le vrai SMTP
Liaison GitLab (tâches ↔ commits)	Fait, non branché	Il manque un jeton `read_api` pour tester sur les vrais dépôts
PostgreSQL / pgAdmin	Prêt	Le code fonctionne avec PostgreSQL. Seul H2 a été réellement utilisé jusqu'ici
Déploiement Docker	Prêt, non déployé	`docker-compose.yml` fourni, aucun serveur cible pour l'instant
Mise en production	À faire	Voir « Reste à faire »

En chiffres : 9 tests automatisés backend, tous verts ; environ 4 000 lignes Java et 2 600 lignes TypeScript ; 9 écrans Angular ; 35 routes d'API.

Fonctionnalités livrées

Chaque rôle a ses écrans. Un PM ou un Head of Office est aussi consultant : il a sa propre semaine et sa propre fiche.

Consultant / architecte

- **Ma semaine** : saisie au fil de l'eau de ce qui a été fait chaque jour (projet, tâche liée ou travail non planifié, description, commentaire). Boutons « + Ajouter » et « ↻ Comme la veille ». Le week-end est masqué par défaut.
- **Suggestions GitLab** : sous chaque jour, les commits pas encore notés dans la fiche sont proposés, regroupés par tâche. Un clic crée la ligne pré-remplie.
- **Mes tâches** : liste des tâches assignées avec statut (À faire, En cours, Bloquée, Terminée), échéance en rouge si dépassée, référence `TS-42` et nombre de commits.
- **Ma fiche de temps** : vue mensuelle du 20 au 19, téléchargement Excel, soumission.
- **Rapport** : soumettre, retirer tant qu'il n'est pas validé, corriger et resoumettre après un renvoi.
- **Mon compte** : changement de mot de passe.

Project Manager (PM)

- **Tâches & assignation** : créer, modifier, supprimer et assigner des tâches. Le consultant reçoit un e-mail.
- **Suivi de l'équipe** : qui a soumis, qui manque, qui est à corriger, en hebdo ou en mensuel. Valider ou renvoyer un rapport hebdo avec commentaire. Excel par consultant ou ZIP de toute l'équipe.
- **Fiche d'un consultant** : détail jour par jour et historique de ses rapports.
- **Activité GitLab** : vue globale des commits de son équipe (voir la section Liaison GitLab).
- **Projets & équipe** : création et modification des projets et des Business Units, rattachement des dépôts GitLab.

Head of Office

- Tout ce que fait le PM, mais sur **tous les consultants**.
- **Seul à valider les fiches mensuelles.** Le PM valide les rapports hebdo.
- Voit en plus, dans Activité GitLab, les auteurs de commits non reconnus.

Administrateur

- Gestion des utilisateurs : nom, matricule (ex. C099), profil affiché sur la fiche, e-mail, rôle, PM de rattachement, compte actif, identités Git.
- Gestion des projets et des Business Units.
- Onglet GitLab : état de la connexion, synchronisation manuelle, rattachement dépôt ↔ projet.

Règles métier appliquées

Règle	Détail
Période hebdo	Du lundi au dimanche
Période mensuelle	Du 20 au 19 (ex. 20 mai → 19 juin = « Mai - Juin 2026 »). Décembre se termine en janvier de l'année suivante
Fuseau horaire	Africa/Dakar pour toutes les dates, quel que soit le serveur
Soumission	Refusée si aucune saisie sur la période
Verrouillage	Un rapport soumis ou validé verrouille les saisies de sa période. « À corriger » les déverrouille
Saisie future	Interdite au-delà de 7 jours
Destinataires (hebdo)	Le PM ; à défaut le Head of Office ; le consultant en copie
Destinataires (mensuel)	Le Head of Office ; le PM et le consultant en copie
Renvoi	Le commentaire est obligatoire et envoyé par e-mail au consultant
Visibilité	Un consultant ne voit que ses données ; un PM, son équipe ; le Head of Office et l'Admin, tout le monde

Fichiers Excel générés

Nom au format `Prénom NOM - C099 - Fiche de temps Mai - Juin 2026.xlsx` (ou « Rapport hebdo »). L'en-tête reprend Prénom(s) & Nom, Matricule, Profil et Période. Il y a ensuite une ligne par jour `Date | Projet | Business Unit | Liste des tâches |`

`Commentaire` et le cadre « Signature Consultant ». Un second onglet, « Suivi des tâches », liste les tâches ouvertes et celles terminées sur la période. Le ZIP d'équipe contient en plus un fichier de synthèse.

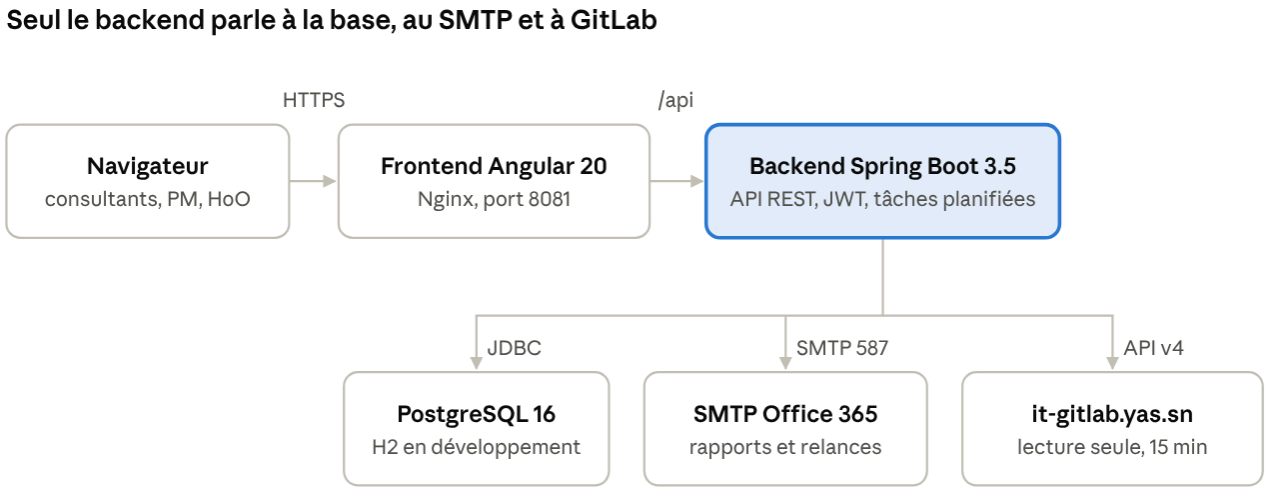
Automatisations (heure de Dakar)

Quand	Quoi
Toutes les 15 min	Synchronisation GitLab (si un jeton est configuré)
Jeudi 16h et vendredi 10h	Relance des consultants qui n'ont pas soumis leur rapport hebdo
Vendredi 18h	Chaque PM reçoit le ZIP des rapports de son équipe et la liste des manquants
Le 18 et le 19, 10h	Relance pour la fiche mensuelle
Le 20, 8h	Le Head of Office reçoit le ZIP de toutes les fiches de la période écoulée et une synthèse

Les horaires se modifient dans `application.yml` (rubrique `app.schedule`).

Architecture technique

L'application a deux parties : un backend Spring Boot qui expose une API REST sécurisée par JWT, et un frontend Angular servi par Nginx. Elle parle à trois systèmes externes : la base de données, le serveur SMTP et GitLab.



architecture · 3 couches, 3 systèmes externes

Le navigateur ne parle qu'au frontend, qui relaie `/api` vers le backend. La base, l'envoi d'e-mails et GitLab ne sont joints que par le backend.

Stack

Couche	Technologie	Version
Backend	Spring Boot (Web, Security, Data JPA, Validation, Mail), `@Scheduled`	3.5, Java 21
Authentification	JWT sans état, mots de passe BCrypt, expiration 12 h	—
Excel	Apache POI	—
Base de données	H2 fichier en développement, PostgreSQL en production	PostgreSQL 16 (Docker)
Schéma	Généré par Hibernate (`ddl-auto: update`)	—
Client GitLab	Spring `RestClient` , API GitLab v4	—
Frontend	Angular standalone, signals, sans bibliothèque UI externe, responsive	Angular 20, Node 20+
Déploiement	Docker Compose : `db` , `backend` , `frontend` (Nginx, port 8081)	—

Organisation du code backend

Package racine `sn.yas.itfactory.timesheet` :

Package	Contenu
`domain`	Entités JPA : `AppUser` , `Project` , `Task` , `WorkEntry` , `Report` , `GitRepo` , `GitCommit` , `GitMergeRequest` et les énumérations
`repository`	Interfaces Spring Data
`service`	Règles métier : saisies, tâches, fiches, rapports, planificateur, e-mail, périodes, visibilité, GitLab
`gitlab`	Interface `GitLabApi` et son implémentation REST
`excel`	Génération de la fiche YAS et du fichier de synthèse

Package	Contenu
`security`	Filtre JWT, configuration Spring Security, CORS
`web`	Contrôleurs REST, DTO, gestion des erreurs (ProblemDetail)
`config`	Propriétés `app.*`, données de démonstration

Organisation du code frontend

Dossier	Contenu
`core`	Service d'API, authentification, garde de routes, intercepteur JWT, modèles, dates
`pages`	Connexion, Ma semaine, Ma fiche de temps, Tâches, Suivi de l'équipe, Fiche consultant, Activité GitLab, Administration, Mon compte
`shared`	Éditeur de saisie, panneau de rapport, tableau de fiche, panneau GitLab d'une tâche

En développement, le serveur Angular (port 4300) relaie `/api` vers le backend (port 8080) grâce à `proxy.conf.json`.

Modèle de données

Table	Rôle	Liens principaux
`app_user`	Utilisateur : nom, matricule, profil, e-mail, rôle, actif, identités Git	PM (→ `app_user`)
`project`	Projet et Business Unit	PM (→ `app_user`)
`task`	Tâche assignée, statut, échéance	projet, assigné, créateur
`work_entry`	Une ligne de travail d'une journée	consultant, projet, tâche (facultative)
`report`	Rapport hebdo ou mensuel d'un consultant : statut, commentaires, dates	consultant, validateur
`git_repo`	Dépôt GitLab découvert, dernière synchro	projet (facultatif)

Table	Rôle	Liens principaux
`git_commit`	Commit, jour de travail (heure de Dakar), source de la liaison	dépôt, consultant, tâche
`git_merge_request`	Merge request qui référence une tâche	dépôt, consultant, tâche

Statuts d'un rapport : Non soumis (`BROUILLON`), Soumis, Validé, À corriger (`REJETE`). Rôles : `CONSULTANT` , `PM` , `HEAD_OF_OFFICE` , `ADMIN` .

Liaison GitLab

Chaque tâche porte une référence courte, `TS-42` , que le consultant met dans le nom de sa branche. La synchronisation relie alors chaque commit à une tâche et à un consultant. On obtient ainsi une vue globale par projet et par consultant. Les commits servent de **preuve** du travail et d'aide à la saisie, **jamais de mesure du temps** : l'analyse, les réunions et l'architecture ne produisent pas de commits.

Pourquoi la branche plutôt que chaque commit

Si on compte sur le message de chaque commit, un seul oubli suffit à perdre le lien, et cela arrive souvent. Avec la branche, la référence est écrite une seule fois et tous ses commits sont rattachés. La référence dans le message reste possible pour les commits faits directement sur `develop` ou `main` .

Règles de liaison

Lien	Règle	Priorité
Commit → tâche	`TS-42` dans le **message du commit**	1 (gagne toujours)
Commit → tâche	`TS-42` dans le **titre ou la branche source d'une merge request** : tous les commits de la MR	2
Commit → tâche	`TS-42` dans le **nom d'une branche sans MR** : les commits présents sur la branche et absents de la branche par défaut	3
Commit → consultant	E-mail de l'auteur = e-mail du compte, ou une des « Identités Git » de l'utilisateur	—
MR → consultant	Nom d'utilisateur GitLab de l'auteur présent dans les « Identités Git »	—

Lien	Règle	Priorité
Dépôt → projet	Choisi dans Administration › GitLab. Automatique à la découverte si le groupe porte le nom du projet (`numflex/*` → NumFlex)	—

La casse est ignorée : `feature/ts-42-export` fonctionne. `ATS-5` n'est pas une référence. Les commits de merge (« Merge branch … ») sont ignorés. Une référence vers une tâche qui n'existe pas est ignorée. Quand une identité Git est ajoutée à un utilisateur, ses anciens commits lui sont rattachés immédiatement.

Déroulement d'une synchronisation

1. **Découverte** des dépôts : ceux des groupes listés dans `GITLAB_GROUPS` (sous-groupes compris), ou à défaut tous ceux dont le propriétaire du jeton est membre. Les dépôts archivés sont exclus.
2. Pour chaque dépôt **rattaché à un projet** (les autres sont ignorés) :
   a. lecture des commits de toutes les branches depuis la dernière synchro, moins 1 jour de recouvrement (60 jours au premier passage) ;
   b. lecture des merge requests mises à jour depuis, puis des commits de celles qui portent une référence ;
   c. lecture des branches qui portent une référence et n'ont pas de MR, puis comparaison avec la branche par défaut.
3. **Enregistrement** en une transaction par dépôt. Les appels GitLab sont faits avant, hors transaction.
4. Une erreur sur un dépôt n'arrête pas les autres. Elle est affichée dans Administration › GitLab et sur la page Activité GitLab.

La synchronisation tourne toutes les 15 minutes. Elle se lance aussi à la main avec le bouton « Synchroniser maintenant ». Elle est **en lecture seule** : rien n'est jamais écrit dans GitLab. Les appels paginés sont limités à 2 000 éléments.

Ce que voit chaque rôle

Écran	Contenu
Ma semaine (consultant)	Les commits du jour pas encore saisis, regroupés par tâche (ou par projet s'ils n'ont pas de tâche). Bouton « + Ajouter » qui pré-remplit la ligne avec les titres des commits

Écran	Contenu
Tâches	La référence `TS-42` et le nombre de commits. Un clic ouvre le panneau : le nom de branche à copier (`feature/TS-42-titre-de-la-tache`), les merge requests (ouverte, fusionnée, fermée), les commits et la façon dont chacun a été lié
Activité GitLab (PM, Head of Office)	Par semaine ou par mois (20 → 19), filtrable par projet : commits, part liée à une tâche, jours avec commits, jours déclarés sur le projet, jours avec commits mais sans aucune ligne dans la fiche
Activité GitLab, écarts	Tâches « En cours » sans commit sur la période (projets suivis seulement), commits sans tâche (100 au plus), auteurs non reconnus (Head of Office et Admin)
Administration › GitLab	État de la connexion, dernière synchro et erreurs, bouton de synchro, liste des dépôts et projet de rattachement

Dépôts connus sur it-gitlab.yas.sn

Relevé du 5 octobre 2026, à partir de la page Explore.

Dépôt	Groupe	Dernière activité	Projet de rattachement proposé
numflex-backend	numflex	il y a 15 heures	NumFlex (automatique)
numflex-backoffice	numflex	il y a 2 jours	NumFlex (automatique)
number-one-backend	NUMBER-ONE	il y a 5 jours	NUMBER-ONE, projet à créer dans l'application
number-one-backoffice	NUMBER-ONE	il y a 5 jours	NUMBER-ONE, projet à créer dans l'application
Loto Dispatch, LOTO Manual Draw	TOMBOLA	il y a 7 mois	À décider
5 dépôts wso2-*	IT-FACTORY / WSO2	il y a 1 an	Probablement non suivis

Référence de l'API REST

Toutes les routes sont sous `/api`. Sauf la connexion, elles exigent l'en-tête `Authorization: Bearer <jeton>`. Les erreurs sont renvoyées au format ProblemDetail, avec un message en français dans `detail`. Les dates sont au format `AAAA-MM-JJ`. Le paramètre `type` vaut `HEBDO` ou `MENSUEL`.

Méthode	Route	Qui	Rôle
POST	`/auth/login`	Public	Connexion, renvoie le jeton et l'utilisateur
GET	`/auth/me`	Tous	Utilisateur connecté
POST	`/auth/password`	Tous	Changer son mot de passe
GET	`/users`	PM, HoO, Admin	Liste des utilisateurs
GET	`/users/team`	Tous	Les consultants que je suis
POST, PUT	`/users`, `/users/{id}`	Admin	Créer ou modifier un utilisateur
GET	`/projects`	Tous	Liste des projets
POST, PUT	`/projects`, `/projects/{id}`	PM, HoO, Admin	Créer ou modifier un projet
GET	`/entries?from=&to=&consultantId=`	Tous (visibilité)	Saisies d'une période
POST, PUT, DELETE	`/entries`, `/entries/{id}`	Le consultant	Gérer ses saisies (refusé si la période est verrouillée)
GET	`/tasks?scope=&assigneeId=&status=&projectId=`	Tous (visibilité)	Tâches, avec référence et nombre de commits

Méthode	Route	Qui	Rôle
POST, PUT, DELETE	`/tasks`, `/tasks/{id}`	Créateur ou manager	Gérer une tâche
PATCH	`/tasks/{id}/status`	Assigné ou manager	Changer le statut
GET	`/tasks/{id}/git`	Tous (visibilité)	Commits, MR et nom de branche d'une tâche
GET	`/reports/timesheet?type=&date=&consultantId=`	Tous (visibilité)	Fiche de la période
POST	`/reports/submit?type=&date=`	Le consultant	Soumettre (envoie l'Excel par e-mail)
POST	`/reports/withdraw?type=&date=`	Le consultant	Retirer une soumission non validée
POST	`/reports/{id}/approve`, `/reports/{id}/reject`	PM (hebdo), HoO (mensuel)	Valider ou renvoyer avec commentaire
GET	`/reports/team?type=&date=`	Managers	Suivi de l'équipe
GET	`/reports/history?consultantId=`	Tous (visibilité)	Historique des rapports
GET	`/reports/excel?type=&date=&consultantId=`	Tous (visibilité)	Télécharger l'Excel
GET	`/reports/team/zip?type=&date=`	Managers	ZIP de l'équipe et synthèse
GET	`/git/status`	Tous	État de la connexion GitLab
POST	`/git/sync`	PM, HoO, Admin	Synchroniser maintenant
GET, PUT	`/git/repos`, `/git/repos/{id}`	PM, HoO, Admin	Dépôts et rattachement à un projet

Méthode	Route	Qui	Rôle
GET	`/git/suggestions?from=&to=&consultantId=`	Tous (visibilité)	Commits pas encore saisis dans la fiche
GET	`/git/activity?type=&date=&projectId=`	PM, HoO, Admin	Vue globale GitLab de l'équipe

HoO = Head of Office. « Visibilité » : chacun pour lui-même, un PM pour son équipe, le Head of Office et l'Admin pour tous. Un manager ne peut pas valider son propre rapport.

Installation et exploitation

Il y a trois façons de lancer l'application : en local avec H2 (aucune installation de base), en local avec PostgreSQL (visible dans pgAdmin), ou en production avec Docker. Prérequis : Java 21+, Maven, Node 20+.

1. En local avec H2 (le plus simple)

```
# Backend sur http://localhost:8080, base dans backend/data
cd backend && mvn spring-boot:run

# Frontend sur http://localhost:4300
cd frontend && npm install && npm start
```

Les e-mails ne partent pas (`MAIL_ENABLED=false`). Ils sont déposés avec leurs pièces jointes dans `backend/data/outbox/` . La console H2 est disponible sur `/h2-console` .

2. En local avec PostgreSQL et pgAdmin

PostgreSQL 18 et pgAdmin 4 sont installés sur le poste de développement (`/Applications/PostgreSQL 18/`).

1. Dans pgAdmin, ouvrir le serveur PostgreSQL 18, puis exécuter dans le Query Tool :

```
CREATE ROLE timesheet LOGIN PASSWORD 'timesheet';
CREATE DATABASE timesheet OWNER timesheet;
```

2. Lancer le backend sur cette base. Les tables sont créées au démarrage :

```
cd backend
DB_URL=jdbc:postgresql://localhost:5432/timesheet \
DB_USER=timesheet DB_PASSWORD=timesheet \
mvn spring-boot:run
```

3. Dans pgAdmin : Databases › timesheet › Schemas › public › Tables, puis clic droit › Refresh.

Les données saisies sur H2 ne sont pas reprises : la base PostgreSQL démarre avec les seuls comptes de démonstration.

3. En production avec Docker

```
cp .env.example .env   # renseigner les secrets
docker compose up -d --build
# → http://<serveur>:8081
```

Trois conteneurs : `db` (PostgreSQL 16, volume `db-data`), `backend` (volume `backend-data`), `frontend` (Nginx, port 8081). Passer `APP_SEED=false` pour ne pas créer les comptes de démonstration, puis créer l'administrateur réel.

Variables d'environnement du backend

Variable	Valeur par défaut	Rôle
`DB_URL`, `DB_USER`, `DB_PASSWORD`	H2 fichier `./data/timesheet`, `sa`	Base de données
`JWT_SECRET`	Valeur de démonstration	**À changer en production**, 32 caractères minimum
`APP_SEED`	`true`	Crée les comptes et données de démo si la base est vide
`FRONTEND_URL`, `CORS_ORIGINS`	`http://localhost:4300`	Adresse du frontend (liens des e-mails, CORS)
`MAIL_ENABLED`	`false`	`true` pour envoyer réellement les e-mails
`MAIL_HOST`, `MAIL_PORT`	`smtp.office365.com`, `587`	Serveur SMTP
`MAIL_USER`, `MAIL_PASSWORD`, `MAIL_FROM`	`no-reply@yas-itfactory.sn`	Boîte d'envoi

Variable	Valeur par défaut	Rôle
`GITLAB_URL`	`https://it-gitlab.yas.sn`	Instance GitLab
`GITLAB_TOKEN`	vide (GitLab désactivé)	Jeton avec le droit `read_api`
`GITLAB_GROUPS`	vide (tous les dépôts du jeton)	Groupes suivis, ex. `numflex,NUMBER-ONE`
`GITLAB_TASK_PREFIX`	`TS`	Préfixe des références de tâche

Brancher GitLab

1. Créer un jeton dans GitLab (avatar › Preferences › Access tokens), droit `read_api` seulement. En production, un jeton de groupe ou un compte de service est préférable à un jeton personnel.
2. Redémarrer le backend avec `GITLAB_TOKEN` et, si besoin, `GITLAB_GROUPS`.
3. Dans Administration › GitLab, cliquer sur « Synchroniser maintenant », puis vérifier le rattachement des dépôts.
4. Demander aux consultants de nommer leurs branches `feature/TS-<numéro>-<titre>`.

Comptes de démonstration

Mot de passe `Yas@2026` pour tous, e-mails en `@yas-itfactory.sn`.

Compte	Rôle	Remarque
`moussa@`	Consultant (C060)	Tâches NumFlex, commits GitLab de démo
`fatou@`	Consultante (C072)	Tâches CRM Entreprise
`cheikh@`	Consultant (C099)	Architecte NumFlex
`pm@`	Project Manager	PM des trois consultants
`head@`	Head of Office	Voit tout, valide les fiches mensuelles
`admin@`	Administrateur	Utilisateurs, projets, GitLab

Les données de démonstration (projets, tâches, saisies des 6 derniers jours, dépôt GitLab fictif) ne sont créées que sur une base vide.

Tests et qualité

Le backend a 9 tests automatisés, tous verts au 5 octobre 2026. Ils se lancent avec `cd backend && mvn test`. Le frontend n'a aucun test automatisé : il est seulement vérifié par sa compilation (`ng build`).

Fichier	Test	Ce qui est vérifié
`PeriodTest`	4 tests	Période mensuelle du 20 au 19, le 20 ouvre une nouvelle période, décembre se termine en janvier, semaine du lundi au dimanche
`ReportFlowTest`	Soumission et validation hebdo	Saisie, soumission, verrouillage, validation par le PM
`ReportFlowTest`	Format Excel mensuel	Le fichier suit le modèle YAS (en-tête, colonnes, onglets)
`ReportFlowTest`	Droits d'accès	Un consultant ne voit pas la fiche d'un autre
`GitSyncTest`	Références de tâche	`TS-42` trouvé dans une branche, un titre ou un message ; `ATS-5` refusé
`GitSyncTest`	Synchronisation complète avec un faux GitLab	Rattachement automatique numflex → NumFlex, dépôt non rattaché jamais interrogé, priorité message > MR > branche, commits de merge ignorés, auteur inconnu, deuxième passage sans doublon, identité Git ajoutée après coup, suggestions, vue d'activité et droits d'accès

Les tests tournent sur une base H2 en mémoire, à l'heure de Dakar.

Vérifications manuelles faites : appels de l'API GitLab de l'application sur une instance de test (port 8090), avec les données de démonstration : références et nombre de commits des tâches, suggestions, vue d'activité PM et Head of Office, refus de la synchronisation sans jeton.

Pas encore testé :

- l'appel réel à it-gitlab.yas.sn (format des réponses, pagination, droits du jeton) ;
- l'envoi réel des e-mails par Office 365 ;

- le fonctionnement sur PostgreSQL (seul H2 a été utilisé) ;
- le déploiement Docker de bout en bout ;
- l'interface Angular, écran par écran, dans un navigateur, après l'ajout de la liaison GitLab ;
- la tenue en charge avec un vrai volume de consultants et de commits.

Reste à faire

Avant la mise en production, il reste 8 chantiers bloquants (P0), surtout de l'exploitation : versionner le code, brancher le vrai GitLab, valider PostgreSQL, configurer les secrets, le SMTP et HTTPS, charger les vrais comptes, mettre en place les sauvegardes. Le code fonctionnel, lui, est prêt. Les colonnes Priorité et Statut sont modifiables directement dans le document.

#	Chantier	Détail	Priorité	Statut
1	Versionner le code	Le dossier du projet n'est pas encore un dépôt Git. Créer le dépôt (par exemple dans it-gitlab.yas.sn), premier commit, branche `develop`	P0 · bloquant prod	À faire
2	Brancher le vrai GitLab	Obtenir un jeton `read_api` (de préférence de groupe ou de compte de service), lancer la première synchro, corriger les écarts éventuels de l'API réelle	P0 · bloquant prod	À faire
3	Valider PostgreSQL	Faire tourner l'application et les tests sur PostgreSQL (pgAdmin), puis remplacer `ddl-auto: update` par des migrations versionnées (Flyway)	P0 · bloquant prod	À faire
4	Secrets de production	`JWT_SECRET` aléatoire (32 caractères minimum), `DB_PASSWORD` fort, aucun secret dans le code ni dans Git	P0 · bloquant prod	À faire

#	Chantier	Détail	Priorité	Statut
5	E-mails Office 365	Boîte d'envoi `no-reply@yas-itfactory.sn`, `MAIL_ENABLED=true`, test réel de la soumission, des relances et des ZIP	P0 · bloquant prod	À faire
6	Serveur et HTTPS	Serveur cible, nom de domaine (ex. timesheet.yas-itfactory.sn), certificat, `FRONTEND_URL` et CORS à jour	P0 · bloquant prod	À faire
7	Comptes réels	`APP_SEED=false`, administrateur réel, saisie ou import des consultants (matricule, profil, PM), des projets et des Business Units	P0 · bloquant prod	À faire
8	Sauvegardes	Sauvegarde quotidienne de PostgreSQL (`pg_dump`) et du volume `backend-data`, test de restauration	P0 · bloquant prod	À faire
9	Recette par rôle	Parcours complet dans le navigateur pour consultant, PM, Head of Office et Admin, écrans GitLab compris	P1 · important	À faire
10	Convention de branches	Communiquer aux équipes le format `feature/TS-<numéro>-<titre>`, confirmer le préfixe `TS`	P1 · important	À faire
11	Projets GitLab	Créer le projet NUMBER-ONE dans l'application et rattacher ses 2 dépôts ; décider pour TOMBOLA et WSO2	P1 · important	À faire
12	Tests frontend	Aucun test Angular aujourd'hui : au minimum la saisie, la soumission et la validation	P1 · important	À faire

#	Chantier	Détail	Priorité	Statut
13	Supervision	La route `/actuator/health` est autorisée mais Spring Actuator n'est pas installé : l'ajouter, centraliser les logs, alerter si la synchro GitLab échoue	P1 · important	À faire
14	Mot de passe oublié	Seuls l'admin (réinitialisation) et l'utilisateur connecté (changement) peuvent changer un mot de passe : ajouter un lien de réinitialisation par e-mail	P1 · important	À faire
15	Congés et jours fériés	Aucune notion d'absence ni de jour férié : ils ne peuvent être notés qu'en commentaire. À trancher avec le Head of Office	P1 · important	À faire
16	Sécurité du jeton JWT	Le jeton est gardé 12 h dans le navigateur (`localStorage`) sans révocation possible : envisager un jeton court avec renouvellement	P1 · important	À faire
17	Webhook GitLab	Mise à jour en temps réel en plus de la synchro toutes les 15 min (il faut que GitLab puisse joindre l'application)	P2 · amélioration	À faire
18	Statut automatique des tâches	Option : « En cours » au premier commit, « Terminée » proposée quand la MR est fusionnée	P2 · amélioration	À faire
19	Connexion Office 365 (SSO)	Remplacer les mots de passe propres à l'application par le compte Microsoft de l'entreprise	P2 · amélioration	À faire

#	Chantier	Détail	Priorité	Statut
20	GitLab dans les exports	Ajouter l'activité GitLab de la période au fichier de synthèse du ZIP mensuel	P2 · amélioration	À faire
21	Performances de la vue Activité	Elle lit les saisies consultant par consultant : à optimiser au-delà de quelques dizaines de consultants	P2 · amélioration	À faire
22	Durée par ligne	Les saisies n'ont pas de durée (heures ou demi-journées). À ajouter seulement si la fiche YAS l'exige	P2 · amélioration	À faire
23	Tableau de bord Head of Office	Taux de soumission dans les délais, retards récurrents, charge par projet	P2 · amélioration	À faire
24	Application mobile ou PWA	Saisie rapide depuis le téléphone ; l'interface est déjà responsive	P2 · amélioration	À faire

Décisions en suspens et risques

Six décisions attendent une validation, dont trois choix faits par défaut pendant le développement. Le principal risque est l'adoption de la convention de branches par les équipes : sans elle, la vue GitLab montre surtout des commits sans tâche.

Décisions à confirmer

- [ ] **Préfixe des tâches** : `TS` retenu par défaut (variable `GITLAB_TASK_PREFIX`). L'équipe a-t-elle déjà une convention de nommage des branches ?
- [ ] **Statut automatique des tâches** : aujourd'hui l'application propose seulement, elle ne change rien seule. Faut-il passer une tâche « En cours » au premier commit et proposer « Terminée » quand la MR est fusionnée ?
- [ ] **Synchro par interrogation ou par webhook** : interrogation toutes les 15 min retenue, car elle fonctionne même si GitLab ne peut pas joindre l'application. Le webhook demande une ouverture réseau.
- [ ] **Jeton GitLab** : jeton personnel, jeton de groupe ou compte de service ? Un jeton personnel s'arrête si son propriétaire quitte YAS ou perd ses droits.

- [ ] **Dépôts suivis** : NUMBER-ONE à créer comme projet ; TOMBOLA et WSO2 suivis ou non ?
- [ ] **Congés et jours fériés** : simple commentaire, comme aujourd'hui, ou vraie notion d'absence sur la fiche ?

Risques et points d'attention

Risque	Conséquence	Parade
Les consultants ne mettent pas `TS-42` dans leurs branches	Beaucoup de commits sans tâche, vue globale peu utile	Bouton « Copier le nom de branche », communication, suivi de la liste « Commits sans tâche »
Commits faits avec un e-mail personnel	Commits non rattachés au consultant	Liste « Auteurs non reconnus » et champ « Identités Git », rattachement rétroactif
Commits lus comme une mesure de la productivité	Malaise des équipes ; rôles sans code (architecte, PM) défavorisés	Présenter GitLab comme une aide à la saisie et une preuve, jamais comme un compteur ; aucune heure calculée à partir des commits
Branche fusionnée avant la synchro, sans MR	Ses commits ne sont plus distinguables de la branche par défaut et restent sans tâche	Passer par une merge request (la liste de ses commits reste disponible après fusion) ou mettre la référence dans le message
Référence `TS-42` basée sur l'identifiant en base	Si la base est recréée, les numéros changent et les anciennes branches pointent vers d'autres tâches	Ne jamais recréer la base de production ; sauvegardes (chantier 8)
Schéma géré par `ddl-auto: update`	Une évolution du modèle peut échouer ou laisser des colonnes orphelines en production	Migrations Flyway avant la mise en production (chantier 3)
Un seul jeton GitLab personnel	La synchro s'arrête au départ de son propriétaire	Jeton de groupe ou compte de service

Risque	Conséquence	Parade
Données personnelles (e-mails, activité individuelle)	Obligations de confidentialité	Accès limité par rôle (déjà en place), HTTPS, informer les consultants de ce qui est collecté